
EATS

Full Stack Food Delivery Application

System Design — Project Report

Submitted By	Ranvendra Pratap Singh
Subject	System Design
Stack	MERN (MongoDB, Express.js, React, Node.js)
Backend Language	TypeScript 6.x — Class-Based OOP
Frontend	React 19 + Vite 7 + Redux Toolkit 2
Database	MongoDB Atlas (5 Collections)
Live URL	<code>eatindia.vercel.app</code>
Date	April 2026

Abstract

Eats is a production-deployed, full-stack food delivery web application built on the MERN stack (MongoDB, Express.js, React, Node.js). The backend is written in TypeScript 6 following a five-layer class-based object-oriented architecture. The frontend is a React 19 Single Page Application powered by Vite 7, Redux Toolkit 2 for centralised state management, and Tailwind CSS 4 for styling. Both the frontend and backend are deployed to production and publicly accessible.

The application allows users to browse restaurants, explore categorised menus, manage a cart that synchronises automatically with the database, simulate a Razorpay payment, and review a complete order history. The system integrates MongoDB Atlas, Cloudinary for image storage, and Razorpay for payment orchestration.

This report is a System Design document covering the high-level deployment architecture, the five-layer backend design, five MongoDB collection schemas, fifteen REST API endpoints, frontend Redux state management, four key data flows, nine software design patterns, all five SOLID principles, cross-browser authentication strategy, security controls, and the production deployment pipeline on Vercel and Render.

Keywords: MERN Stack, TypeScript, OOP, REST API, MongoDB, Redux Toolkit, JWT, Cloudinary, Razorpay, SOLID Principles, Design Patterns.

Contents

Abstract	ii
1 Introduction	2
1.1 What Is Eats?	2
1.2 Motivation	2
1.3 Project Objectives	2
1.4 Scope of This Report	3
2 System Requirements	4
2.1 Functional Requirements	4
2.1.1 Authentication and Profile Management	4
2.1.2 Restaurant Discovery	4
2.1.3 Menu and Cart	4
2.1.4 Orders and Payment	4
2.2 Non-Functional Requirements	5
3 Technology Stack	6
3.1 Backend	6
3.2 Frontend	7
3.3 Cloud and External Services	7
4 System Architecture	8
4.1 High-Level Architecture	8
4.2 Backend Architecture — Five-Layer Design	8
4.2.1 Routes Interface Contract	9
4.3 Frontend Architecture	9
5 Database Design	11
5.1 Database Choice	11
5.2 Entity Relationship Diagram	11
5.3 Collection Schemas	12
5.3.1 User	12
5.3.2 Restaurant	13
5.3.3 MenuItem	13
5.3.4 Cart	14

5.3.5	Order	14
6	REST API Design	16
6.1	API Conventions	16
6.2	Authentication — /api/v1/auth	16
6.3	Restaurants — /api/v1/restaurants	17
6.4	Cart — /api/v1/cart	17
6.5	Orders — /api/v1/orders	17
6.6	Payment — /api/v1/payment	18
7	Key Data Flows	19
7.1	Login and Session Initialisation	19
7.2	Cart Synchronisation	19
7.3	Checkout and Order Placement	19
7.4	Session Restoration on Page Refresh	20
8	Frontend Architecture	22
8.1	Redux Store — Three Slices	22
8.1.1	The isHydrated Flag	22
8.2	Key Components	22
9	Design Patterns	24
9.1	1. Facade Pattern	24
9.2	2. Dependency Injection	25
9.3	3. Chain of Responsibility	25
9.4	4. Strategy Pattern	26
9.5	5. Observer Pattern	26
9.6	6. Proxy Pattern	27
9.7	7. Memento / Snapshot Pattern	27
9.8	8. Module Pattern	28
9.9	9. Singleton Pattern	28
10	SOLID Principles	29
10.1	S — Single Responsibility Principle	29
10.2	O — Open/Closed Principle	29
10.3	L — Liskov Substitution Principle	30
10.4	I — Interface Segregation Principle	30
10.5	D — Dependency Inversion Principle	30
11	Authentication Design	32
11.1	The Cross-Browser Problem	32

11.2	Solution: localStorage with the Authorization Header	32
11.3	Security Controls	33
12	Deployment	34
12.1	Deployment Topology	34
12.2	Frontend — Vercel	34
12.3	Backend — Render	35
12.4	Environment Variables	35
13	Limitations and Future Scope	36
13.1	Current Limitations	36
13.2	Future Enhancements	36
14	Conclusion	37
14.1	Summary	37
14.2	Key Engineering Decisions	38
14.3	Learning Outcomes	38
	References	39

List of Figures

4.1	High-Level System Architecture	8
4.2	Backend Five-Layer Architecture	9
5.1	Entity Relationship Diagram	11
12.1	Production Deployment Architecture	34

List of Tables

2.1	Non-Functional Requirements	5
3.1	Backend Technology Stack	6
3.2	Frontend Technology Stack	7
3.3	Cloud and External Services	7
4.1	Backend Architecture Layers	8
5.1	User Collection Schema	12
5.2	Restaurant Collection Schema	13
5.3	MenuItem Collection Schema	13
5.4	Cart Collection Schema	14
5.5	Order Collection Schema	15
6.1	Authentication Endpoints	16
6.2	Restaurant Endpoints	17
6.3	Cart Endpoints	17
6.4	Order Endpoints	17
6.5	Payment Endpoints	18
8.1	Redux Store Slices	22
9.1	Design Patterns Summary	24
9.2	Singleton Instances in Eats	28
10.1	SOLID Principles at a Glance	29
10.2	SRP: One Responsibility Per File	29
11.1	Authentication Solution Components	32
11.2	Security Controls	33
12.1	Vercel Configuration	34
12.2	Render Configuration	35
12.3	Server Environment Variables (server/.env)	35
13.1	Known Limitations	36
13.2	Planned Enhancements	36
14.1	Engineering Decisions and Their Rationale	38

1 Introduction

1.1 What Is Eats?

Eats is a full-stack food delivery web application, conceptually similar to Swiggy or Zomato. It is a complete, deployed system. Users can browse restaurants, explore menus organised by category, add food items to a shopping cart, pay through a Razorpay-styled checkout interface, and review their complete order history.

The frontend is served at `eatindia.vercel.app` (Vercel). The backend REST API runs on Render. All data is persisted in MongoDB Atlas.

1.2 Motivation

Food delivery platforms exercise nearly every fundamental concept in modern software engineering, making them an ideal context for demonstrating comprehensive technical skills:

1. **User Authentication** — Registration, login, JWT, password hashing
2. **Database Design** — Multiple related collections, embedded documents, compound indexes
3. **REST API Design** — HTTP methods, status codes, protected routes, pagination
4. **State Management** — Complex, cross-feature frontend state (cart, user, restaurants)
5. **File Uploads** — Profile pictures streamed to Cloudinary without temporary disk writes
6. **Payment Integration** — Razorpay order creation and HMAC-SHA256 signature verification
7. **Performance UX** — Lazy image loading, skeleton screens, debounced sync
8. **Production Deployment** — Environment variables, CI/CD pipelines, cloud services

1.3 Project Objectives

1. Build and deploy a complete MERN stack application
2. Implement the backend in TypeScript with a five-layer class-based OOP architecture
3. Apply all five SOLID design principles with verifiable code evidence
4. Apply nine classical software design patterns, each solving a specific problem
5. Diagnose and fix the cross-browser authentication failure caused by Safari ITP
6. Write documentation covering every significant architectural decision

1.4 Scope of This Report

This document is a System Design report. It covers:

- **High-Level Design (HLD):** Deployment topology, technology stack, layer overview
- **Low-Level Design (LLD):** Database schemas, API contracts, class structures, data flow sequences
- **Cross-cutting Concerns:** Design patterns, SOLID principles, authentication strategy, security

2 System Requirements

2.1 Functional Requirements

Functional requirements specify what the system must do.

2.1.1 Authentication and Profile Management

- FR-01: Create a new account using name, email, phone number, and password
- FR-02: Log in using either email address or phone number as the identifier
- FR-03: Generate a signed JWT on successful login; return it in the response body
- FR-04: JWT expires automatically after 7 days
- FR-05: View and update profile fields (name, address, city, nickname, gender, country, language, timezone)
- FR-06: Upload a profile picture; store on Cloudinary and serve via CDN
- FR-07: Log out; remove token from both client storage and the server cookie

2.1.2 Restaurant Discovery

- FR-08: Display a paginated list of restaurants (20 per page)
- FR-09: Client-side filter by name, cuisine, minimum rating, and veg-only preference
- FR-10: Client-side sort by rating, delivery time, or cost
- FR-11: Restaurant cards display: image, name, cuisines, rating, delivery time, and active offer

2.1.3 Menu and Cart

- FR-12: Browse a restaurant's full menu, grouped by category with a sidebar navigator
- FR-13: Add items via a two-step flow: select quantity, then confirm
- FR-14: Cart is scoped to one restaurant at a time
- FR-15: Cart synchronises automatically with MongoDB (1-second debounced)
- FR-16: Cart persists across page refreshes for authenticated users

2.1.4 Orders and Payment

- FR-17: Checkout via a Razorpay-styled interface (Cards, UPI, Netbanking, Wallets tabs)
- FR-18: Order record stored as a permanent snapshot of all item details at purchase

time

- FR-19: View complete order history, sorted newest first
- FR-20: Orders placed within 60 seconds show a live countdown timer and progress bar
- FR-21: Backend endpoints for Razorpay order creation and HMAC-SHA256 payment verification

2.2 Non-Functional Requirements

Table 2.1: Non-Functional Requirements

Category	Requirement
Security	Passwords hashed with Bcrypt (10 salt rounds). JWTs signed with a secret key. Razorpay payments verified via HMAC-SHA256.
Performance	Images lazy-loaded via <code>IntersectionObserver</code> . Cart sync debounced (1 s). API responses paginated (20 per page).
Compatibility	Authentication works identically across Chrome, Safari, Firefox, and Arc, including cross-domain deployments.
Scalability	New routes and controllers added without modifying the existing <code>App</code> class (Open/Closed Principle).
Maintainability	SOLID principles throughout. Every file has a single, clearly defined responsibility.
Data Integrity	Unique indexes on user email and phone. Invalid <code>ObjectIds</code> rejected with HTTP 400. Order data immutable once saved.
Availability	Deployed on Vercel (frontend) and Render (backend) with automatic restarts and TLS.

3 Technology Stack

3.1 Backend

Table 3.1: Backend Technology Stack

Technology	Version	Purpose and Rationale
Node.js	LTS	Server-side JavaScript runtime
Express.js	5.x	Minimal HTTP framework for routing, middleware, and API responses
TypeScript	6.x	Adds static typing and OOP constructs to JavaScript
MongoDB Atlas	Cloud	NoSQL document database; flexible schema; managed cloud hosting
Mongoose	9.x	ODM for schema definition, validation, and MongoDB query API
JSON Web Token	9.x	Stateless authentication; token carries the user ID; no server session needed
Bcrypt	6.x	One-way password hashing with salting
Cloudinary	2.x	Cloud image storage and CDN; auto-resizes profile pictures at upload time
Multer	2.x	Parses <code>multipart/form-data</code> ; stores files in memory (no disk writes)
Streamifier	0.1.x	Converts an in-memory buffer to a stream for Cloudinary's upload API
Razorpay	2.x	Payment gateway SDK for order creation and HMAC-SHA256 verification
Validator.js	13.x	Email format validation and password strength enforcement
CORS	2.x	Controls which frontend origins may call the backend
Cookie-Parser	1.x	Parses HTTP cookies; required for the auth fallback mechanism
Dotenv	17.x	Loads environment secrets from <code>.env</code> into <code>process.env</code>

3.2 Frontend

Table 3.2: Frontend Technology Stack

Technology	Version	Purpose and Rationale
React	19.x	UI component library
Vite	7.x	Build tool and development server; millisecond startup
React Router DOM	7.x	Client-side routing without full page reloads
Redux Toolkit	2.x	Centralised global state management
Axios	1.x	HTTP client with interceptor support for JWT injection
Tailwind CSS	4.x	Utility-first CSS framework
Framer Motion	12.x	Declarative animation library
Lucide React	0.5x	Lightweight SVG icon library

3.3 Cloud and External Services

Table 3.3: Cloud and External Services

Service	Role
Vercel	Hosts the React SPA; auto-deploys on every Git push; manages HTTPS and CDN
Render	Hosts the Node.js/Express backend API; runs the pre-compiled JavaScript binary
MongoDB Atlas	Managed cloud MongoDB; automatic backups and network access control
Cloudinary	Image storage and transformation; 400×400 px face-crop applied at upload
Razorpay	Payment gateway (test mode); order creation and cryptographic verification

4 System Architecture

4.1 High-Level Architecture

The system is divided into three parts: a React SPA (Vercel), an Express REST API (Render), and a data layer (MongoDB Atlas and Cloudinary).

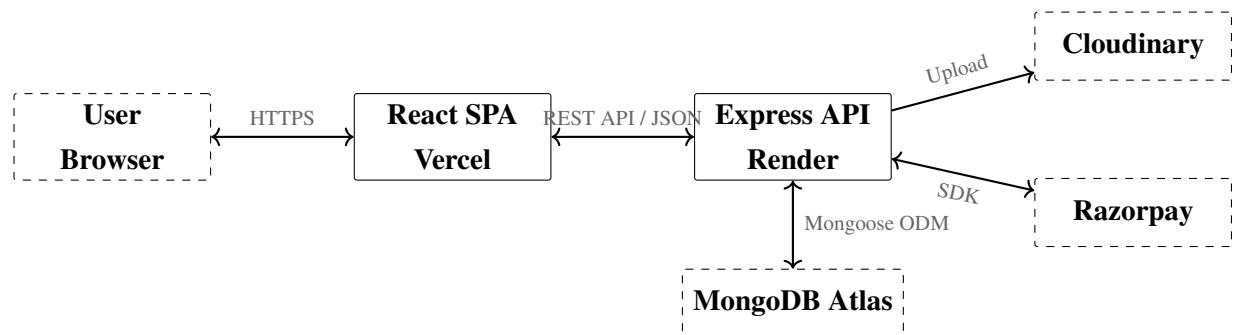


Figure 4.1: High-Level System Architecture

4.2 Backend Architecture — Five-Layer Design

The backend enforces a five-layer class-based architecture. Each layer communicates only with the layer immediately below it, and each has a single, clearly defined purpose.

Table 4.1: Backend Architecture Layers

Layer	File(s)	Responsibility
Entry Point	<code>server.ts</code>	Instantiates App with all route classes; calls <code>startServer()</code>
Application	<code>app.ts</code>	Bootstraps Express: middleware, routes, database connection, HTTP listener
Routes	<code>*.routes.ts</code> (5 files)	Defines URL paths, applies middleware chains, wires to controllers
Controllers	<code>*.controller.ts</code> (5)	Accepts HTTP requests; calls services or models; sends responses
Services	<code>auth.service.ts</code>	Business logic and database queries; no HTTP awareness
Models	<code>models/*.ts</code> (5 files)	Mongoose schemas, TypeScript interfaces, instance methods

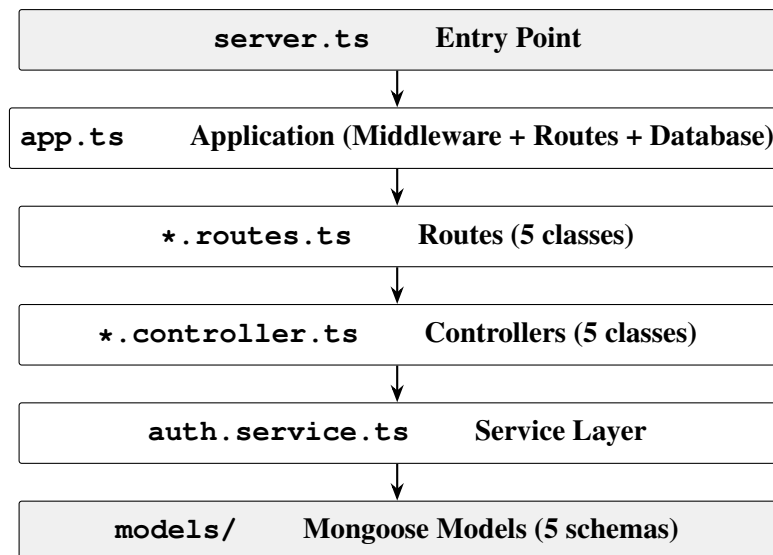


Figure 4.2: Backend Five-Layer Architecture

4.2.1 Routes Interface Contract

Every route class implements the `Routes` interface defined in `utils/route.interface.ts`:

```

1 import { Router } from "express";
2
3 // Every route class must satisfy this two-field contract.
4 export interface Routes {
5   path?: string; // optional base path, e.g. "/api/v1/auth"
6   router: Router; // the Express Router containing all
7                   endpoints
8 }

```

Listing 4.1: Routes Interface (`utils/route.interface.ts`)

The `App` class accepts `Routes[]` and mounts each router generically. It has no knowledge of `AuthRoutes` or `CartRoutes` as concrete types — only the interface. This is the foundation of the Open/Closed and Dependency Inversion SOLID principles.

4.3 Frontend Architecture

The frontend is a Single Page Application. The browser loads one HTML file; all navigation and rendering is managed by client-side JavaScript without full page reloads.

The source tree is organised by feature domain:

- `api/` — `axiosInstance.js` (JWT interceptor singleton) and `authApi.js` (auth module)
- `utils/` — Redux store, three slices (user, cart, restaurant)

- HomePage/, Restaurants/, Cart/, Orders/, Profile/, authPage/
- Toast/, Shimmer/, LazyLoading/, Animations/

5 Database Design

5.1 Database Choice

MongoDB stores data as JSON-like documents inside collections. It was selected because:

- Restaurant menus vary in structure; a flexible document model handles this naturally
- Embedding cart items inside the cart document enables single-document reads
- MongoDB Atlas provides managed cloud hosting with automatic backups and scaling
- Mongoose adds a TypeScript-compatible schema layer with built-in validation

5.2 Entity Relationship Diagram

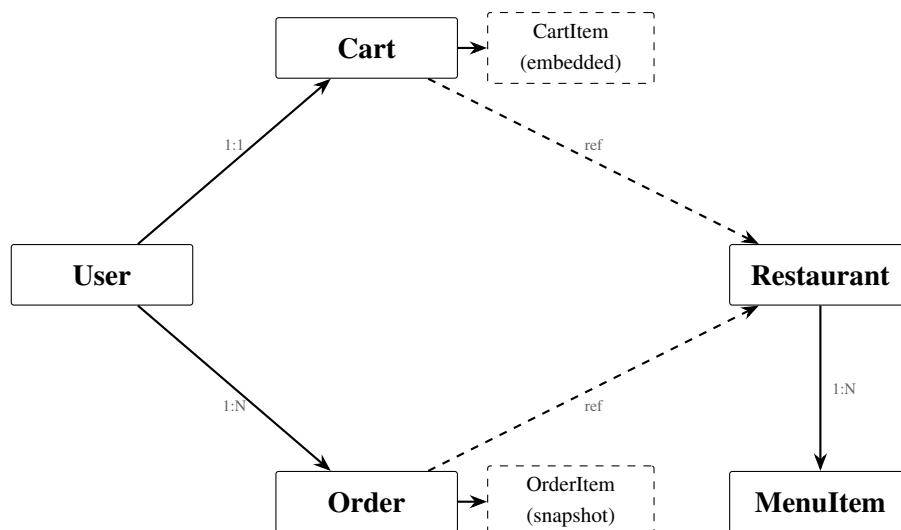


Figure 5.1: Entity Relationship Diagram

5.3 Collection Schemas

5.3.1 User

Table 5.1: User Collection Schema

Field	Type	Constraints and Notes
<code>_id</code>	ObjectId	Auto-generated primary key
<code>userName</code>	String	Required. 3–50 characters.
<code>userEmail</code>	String	Required. Unique. Lowercase. Regex-validated.
<code>password</code>	String	Required. ≥ 8 chars. Stored as Bcrypt hash (10 rounds). Never plain text.
<code>userPhone</code>	String	Required. Unique. 10–15 digits.
<code>userAddress</code>	String	Optional. Default: empty string.
<code>userCity</code>	String	Optional. Default: empty string.
<code>nickName</code>	String	Optional. Default: empty string.
<code>gender</code>	String	Optional. Default: empty string.
<code>country</code>	String	Optional. Default: empty string.
<code>language</code>	String	Optional. Default: empty string.
<code>timeZone</code>	String	Optional. Default: empty string.
<code>profilePicture</code>	String	Cloudinary CDN URL. Default: empty.
<code>createdAt</code>	Date	Auto-managed by Mongoose <code>timestamps</code> option.
<code>updatedAt</code>	Date	Auto-updated on every save.

Instance methods:

- `validatePassword(input)` — runs `bcrypt.compare(input, hash)` and returns a boolean
- `getJWT()` — calls `jwt.sign({_id}, secret, {expiresIn: '7d'})`
- **Pre-save hook** — hashes the password before every save, only when the password field has been modified

5.3.2 Restaurant

Table 5.2: Restaurant Collection Schema

Field	Type	Constraints and Notes
restaurantName	String	Required. Max 100 characters.
restaurantAddress	String	Required. Max 300 characters.
restaurantCity	String	Required. Indexed (single field).
restaurantCuisine	String[]	Required array, e.g. ["Indian", "Chinese"].
restaurantRating	Number	0–5. Default: 0.
restaurantDeliveryTime	Number	Required. In minutes.
restaurantMinOrder	Number	Default: 0.
isRestaurantOpen	Boolean	Default: true.
restaurantImage	String	Required. Cloudinary URL.
isRestaurantPromoted	Boolean	Default: false.
offer	String	Optional discount description.
restaurantTags	String[]	Optional. Default: empty array.

Compound index: { restaurantCity: 1, restaurantCuisine: 1 } allows MongoDB to satisfy city-and-cuisine filter queries without a full collection scan.

5.3.3 MenuItem

Table 5.3: MenuItem Collection Schema

Field	Type	Constraints and Notes
restaurantId	ObjectId	Reference to Restaurant. Required. Indexed.
menuItemName	String	Required. Max 100 characters.
menuItemPrice	Number	Required. Minimum 0.
menuItemCategory	String	Required. Used for menu grouping (e.g. Starters, Main Course).
isMenuItemVeg	Boolean	Default: true.
isMenuItemAvailable	Boolean	Default: true.
menuItemImage	String	Cloudinary URL. Default: empty.
menuItemDescription	String	Optional. Max 200 characters.
menuItemRating	Number	0–5. Default: 0.
menuItemCalories	Number	Optional. Default: null.
menuItemServes	String	Serving size description. Default: "1".

Compound index: { restaurantId: 1, menuItemCategory: 1 } enables efficient category-filtered menu fetches.

5.3.4 Cart

Each user has exactly one cart document, enforced by a unique index on `userId`. Cart items are embedded subdocuments.

Table 5.4: Cart Collection Schema

Field	Type	Constraints and Notes
<code>userId</code>	ObjectId	Reference to User. Required. Unique index.
<code>restaurantId</code>	ObjectId	Reference to Restaurant.
<code>restaurantName</code>	String	Cached restaurant name. Default: null.
<code>items</code>	CartItem[]	Embedded array of CartItem subdocuments.
<code>totalQuantity</code>	Number	Total item count. Default: 0.
<code>totalAmount</code>	Number	Total price. Default: 0.

CartItem subdocument fields: `menuItemId` (ObjectId), `menuItemName` (String), `menuItemPrice` (Number), `itemQuantity` (Number, min 1), `menuItemImage` (String), `isMenuItemVeg` (Boolean). No `_id` on subdocuments.

5.3.5 Order

Order records are immutable snapshots of a transaction.

Table 5.5: Order Collection Schema

Field	Type	Constraints and Notes
userId	ObjectId	Reference to User. Required. Indexed.
restaurantId	ObjectId	Reference to Restaurant. Required. Indexed.
restaurantName	String	Snapshot of the restaurant name at order time.
orderItems	OrderItem[]	Embedded item snapshots (name, price, veg flag captured at purchase).
orderTotalAmount	Number	Required. Minimum 0.
deliveryFee	Number	Default: 49 (INR).
deliveryAddress	String	User's delivery address at order time.
paymentStatus	Enum	PENDING PAID FAILED REFUNDED
orderStatus	Enum	CREATED ACCEPTED PREPARING OUT_FOR_DELIVERY DELIVERED CANCELLED
createdAt	Date	Auto-managed; used for descending sort in order history.

Compound index: { userId: 1, createdAt: -1 } retrieves a user's order history sorted newest-first in a single index scan.

Snapshot principle: OrderItem embeds itemName, itemPrice, and isVeg as copies at purchase time. Future changes to a menu item — renames or price updates — do not affect existing order records. This is an application of the Memento design pattern (Chapter 9).

6 REST API Design

6.1 API Conventions

- **Base prefix:** all routes begin with `/api/v1`
- **Data format:** JSON for all requests and responses; `multipart/form-data` for file uploads
- **Authentication:** protected routes require `Authorization: Bearer <token>`; falls back to an HTTP cookie
- **Methods:** GET (read), POST (create), PUT (replace), PATCH (partial update), DELETE (remove)
- **Status codes:** 200 OK, 201 Created, 400 Bad Request, 401 Unauthorised, 404 Not Found, 500 Server Error

6.2 Authentication — `/api/v1/auth`

Table 6.1: Authentication Endpoints

Method	Path	Auth?	Description
POST	<code>/signup</code>	No	Creates account. Validates email, phone, strong password. Returns user (no password). Status 201.
POST	<code>/login</code>	No	Validates credentials (email or phone + password). Returns user and JWT. Sets cookie as fallback.
POST	<code>/logout</code>	No	Clears the server cookie. Client removes token from <code>localStorage</code> .
GET	<code>/profile</code>	Yes	Returns the authenticated user's profile (password field excluded).
PUT	<code>/profile</code>	Yes	Updates profile fields; accepts <code>multipart/form-data</code> ; uploads picture to Cloudinary.

6.3 Restaurants — `/api/v1/restaurants`

Table 6.2: Restaurant Endpoints

Method	Path	Auth?	Description
GET	/	No	Paginated list (<code>?page=1&limit=20</code>). Returns data and pagination metadata.
GET	<code>/:resId</code>	No	Single restaurant by <code>ObjectId</code> . 400 for invalid ID; 404 if not found.
GET	<code>/:resId/menu</code>	No	All menu items for the specified restaurant.

6.4 Cart — `/api/v1/cart`

Table 6.3: Cart Endpoints

Method	Path	Auth?	Description
GET	/	Yes	Returns the user's cart. Creates an empty cart if none exists (upsert).
POST	<code>/sync</code>	Yes	Overwrites the database cart with the current Redux state (upsert).
DELETE	/	Yes	Resets the user's cart to empty in the database.

6.5 Orders — `/api/v1/orders`

Table 6.4: Order Endpoints

Method	Path	Auth?	Description
POST	/	Yes	Places order; enriches items from DB; saves with <code>paymentStatus: PAID</code> .
GET	/	Yes	Returns all user orders, newest first; populates restaurant details.
PATCH	<code>/:id/status</code>	Yes	Updates a specific order's status; validated against the enum.

6.6 Payment — `/api/v1/payment`

Table 6.5: Payment Endpoints

Method	Path	Auth?	Description
POST	<code>/create-order</code>	No	Creates Razorpay order; converts rupees to paise ($\times 100$). Returns order ID.
POST	<code>/verify-payment</code>	No	Verifies HMAC-SHA256 signature from Razorpay to confirm payment authenticity.

7 Key Data Flows

7.1 Login and Session Initialisation

1. User submits login form (email or phone + password)
2. `validateLoginData(req)` confirms both fields are non-empty
3. `AuthService.loginUser()` queries `User.findOne({$or: [email, phone]})`
4. `user.validatePassword(input)` calls `bcrypt.compare(input, hash)`
5. `user.getJWT()` calls `jwt.sign({_id}, secret, {expiresIn: '7d'})`
6. Server responds with `{user, token}` in the body; also sets an HTTP-only cookie
7. Client stores the token: `localStorage.setItem("eats_token", token)`
8. `loginSuccess(userData)` dispatched to Redux; Navbar and UI update accordingly
9. `GET /api/v1/cart` called immediately; the saved cart is loaded into Redux

7.2 Cart Synchronisation

1. User clicks Add on a menu item card
2. If not authenticated, the AuthSidebar opens instead (protected action)
3. The user adjusts quantity with a local counter and clicks Confirm
4. `addItemToCart()` dispatched; Redux updates `items[]`, totals, and sets `isHydrated: true`
5. `appStore.subscribe()` callback fires (Observer pattern — see Chapter 9)
6. A 1-second debounce timer starts, preventing a database write on every click
7. After 1 second, `POST /api/v1/cart/sync` sends the full cart state
8. MongoDB upserts the user's cart document

7.3 Checkout and Order Placement

1. User opens CartDrawer and reviews items, delivery fee, and total
2. Clicks Checkout Securely; DummyCheckout modal opens with four payment tabs
3. User selects a payment method and clicks Pay
4. A 2-second simulated payment delay shows a loading indicator

5. `POST /api/v1/orders` called with the order payload
6. Server fetches each menu item from MongoDB to create enriched snapshots
7. Order saved with `paymentStatus = "PAID"` and `orderStatus = "PREPARING"`
8. `clearCart()` dispatched; Redux cart becomes empty
9. Observer fires; the logout guard detects this is not a logout transition and skips the sync
10. OrderSuccess animation plays; CartDrawer closes automatically

7.4 Session Restoration on Page Refresh

When the user refreshes the browser, React re-mounts and Redux resets to its default empty state. The application restores the session silently:

1. Redux resets: `isAuthenticated: false`, cart empty
2. `App.jsx` mounts; the initial `useEffect` runs
3. Token read from `localStorage.getItem("eats_token")`
4. Axios interceptor attaches `Authorization: Bearer <token>` to the outgoing request
5. `GET /api/v1/auth/profile` called silently (no login screen shown)
6. Backend `userAuth` middleware verifies the JWT; returns user data
7. `loginSuccess(userData)` dispatched; Redux user state is restored
8. `GET /api/v1/cart` called; the persisted cart is fetched
9. `loadCart(savedCart)` dispatched; Redux cart restored from the database
10. Page finishes loading; the user appears fully logged in with their saved cart

Race Condition Guard: A `useRef` named `initialLoadDone` prevents two concurrent cart fetches on page load — one from the mount effect and one from a second effect watching `isAuthenticated`. The `useRef` ensures only the first fetch runs.

Logout Cart Guard

When the user logs out, `clearCart()` empties the Redux cart. Without a guard, the Observer would sync this empty state to MongoDB and erase the user's saved cart. A transition check prevents this:

```
1 let previousAuthState = false;
2
3 appStore.subscribe(() => {
4   const state           = appStore.getState();
```

```
5   const isAuthenticated = state.user?.isAuthenticated;
6
7   // Detect logout transition (true -> false) and bail out.
8   if (previousAuthState === true && isAuthenticated === false)
9   {
10    previousAuthState = false;
11    clearTimeout(syncTimeout); // cancel any queued sync
12    return; // do NOT write the empty cart
13    to DB
14  }
15  previousAuthState = isAuthenticated;
16
17  if (isAuthenticated && state.cart?.isHydrated) {
18    clearTimeout(syncTimeout);
19    syncTimeout = setTimeout(() => {
20      axiosInstance.post('/api/v1/cart/sync', state.cart);
21    }, 1000); // debounce: 1 second
22  }
23  });
```

Listing 7.1: Logout Guard in `appStore.subscribe()`

8 Frontend Architecture

8.1 Redux Store — Three Slices

Redux provides a single, centralised state container. All components read data via `useSelector` and dispatch actions via `useDispatch`, eliminating prop drilling across the component tree.

Table 8.1: Redux Store Slices

Slice	State	Key Actions
<code>userSlice</code>	<code>userInfo</code> , <code>isAuthenticated</code> , <code>isInitialized</code> , <code>isAuthSidebarOpen</code>	<code>loginSuccess</code> , <code>logoutUser</code> , <code>setAuthInitialized</code>
<code>cartSlice</code>	<code>items[]</code> , <code>totalQuantity</code> , <code>totalAmount</code> , <code>restaurantId</code> , <code>restaurantName</code> , <code>isHydrated</code>	<code>addItemToCart</code> , <code>removeItemFromCart</code> , <code>clearCart</code> , <code>loadCart</code>
<code>restaurantSlice</code>	<code>restaurants[]</code>	<code>setRestaurants</code>

8.1.1 The `isHydrated` Flag

`isHydrated` distinguishes the initial empty Redux default state (before the database response arrives) from a cart that has been actively populated:

- `isHydrated: false` — do not sync to the database (prevents overwriting the user's saved cart)
- `isHydrated: true` — the cart was loaded from the database or the user added an item; safe to sync

8.2 Key Components

Navbar.jsx — Appears on all pages. Shows the logo, navigation links, a cart icon with an item-count badge (`totalQuantity` from Redux), and auth controls (Login button for guests, profile avatar for authenticated users).

Restaurants.jsx — Fetches restaurants once and stores them in Redux. All filtering by

name, cuisine, rating, and veg preference, plus sorting, executes entirely in the browser using `useMemo`. No additional API calls are made when filters change.

MenuItemCard.jsx — Clicking Add shows an inline quantity picker. The item is committed to the Redux cart only after the user clicks Confirm, preventing accidental additions.

LazyImage.jsx — Uses the browser's native `IntersectionObserver` API. Images load only when they enter the viewport, significantly reducing initial page load time on listing pages.

DummyCheckout.jsx — Presents a Razorpay-styled modal with four tabs (Cards, UPI, Netbanking, Wallets). Payment is simulated with a 2-second delay; the order is then written to MongoDB.

OrderHistory.jsx — Orders placed within 60 seconds are classified as active and display a real-time countdown using `setInterval` and an animated progress bar.

9 Design Patterns

Design patterns are proven, reusable solutions to common software design problems. They are templates for structuring code, not ready-made implementations. This chapter documents all nine patterns applied in Eats.

Table 9.1: Design Patterns Summary

Pattern	Category	Where Applied
Facade	Structural	<code>app.ts</code> hides Express bootstrap complexity
Dependency Injection	Creational	<code>server.ts</code> injects routes; <code>AuthController</code> injects <code>AuthService</code>
Chain of Responsibility	Behavioral	Express middleware chains in all <code>*.routes.ts</code>
Strategy	Behavioral	Runtime token source selection in <code>userAuth.ts</code>
Observer	Behavioral	<code>appStore.subscribe()</code> auto-syncs cart to database
Proxy	Structural	<code>axiosInstance.js</code> transparently injects JWT header
Memento / Snapshot	Behavioral	Order item state captured in <code>order.controller.ts</code>
Module	Structural	<code>authApi.js</code> groups all auth calls; hides <code>TOKEN_KEY</code>
Singleton	Creational	<code>axiosInstance</code> , Redux store, MongoDB connection

9.1 1. Facade Pattern

Definition: A Facade provides a simple interface to a complex subsystem.

Starting an Express server involves creating the application object, attaching middleware, registering routes, connecting MongoDB, and opening the HTTP port. The `App` class encapsulates all of this. `server.ts` is just two lines:

```
1 const app = new App([
2   new AuthRoutes(), new CartRoutes(), new OrderRoutes(),
3   new PaymentRoutes(), new RestaurantRoutes(),
```

```
4  });  
5  app.startServer(); // All complexity hidden inside the App  
   class
```

Listing 9.1: Facade Pattern in server.ts

9.2 2. Dependency Injection

Definition: A class receives its dependencies from outside rather than creating them internally.

```
1  // App.ts depends on the Routes interface, not on concrete  
   classes  
2  constructor(routes: Routes[]) {  
3    this.initializeRoutes(routes); // Works with any Routes[]  
       supplied  
4  }  
5  
6  // AuthController receives AuthService as a composed dependency  
7  class AuthController {  
8    private authService = new AuthService(); // injected  
9  
10   public handleLogin = async (req: Request, res: Response) => {  
11     const { user, token } = await  
       this.authService.loginUser(id, pw);  
12     res.json({ data: user, token }); // no bcrypt or JWT  
       knowledge here  
13   };  
14 }
```

Listing 9.2: Dependency Injection

9.3 3. Chain of Responsibility

Definition: A request passes through a sequence of handlers; each either processes it or forwards it.

```
1  this.router.put(  
2    `${this.path}/profile`,  
3    userAuth, // Link 1:  
       verify JWT  
4    upload.single("profilePicture"), // Link 2:  
       process file
```

```
5   this.authController.handleProfileUpdate,      // Link 3:
      update DB
6 );
7 // Failure at any link halts the chain and sends an error
  response.
```

Listing 9.3: Chain of Responsibility in `auth.routes.ts`

9.4 4. Strategy Pattern

Definition: A family of interchangeable algorithms is defined; the correct one is selected at runtime.

```
1 let token: string | undefined;
2 const authHeader = req.headers.authorization;
3
4 if (authHeader && authHeader.startsWith("Bearer ")) {
5   token = authHeader.split(" ")[1]; // Strategy A: header (all
      browsers)
6 } else {
7   token = req.cookies?.token;        // Strategy B: cookie
      (fallback)
8 }
```

Listing 9.4: Strategy Pattern in `userAuth.ts`

9.5 5. Observer Pattern

Definition: An object (Subject) notifies registered Observers automatically when its state changes.

```
1 appStore.subscribe(() => {                // Observer registered
2   const state = appStore.getState();
3   if (state.user?.isAuthenticated && state.cart?.isHydrated) {
4     clearTimeout(syncTimeout);
5     syncTimeout = setTimeout(() => {
6       axiosInstance.post('/api/v1/cart/sync', state.cart);
7     }, 1000); // debounced Observer reaction
8   }
9 });
```

Listing 9.5: Observer Pattern in `appStore.js`

9.6 6. Proxy Pattern

Definition: A Proxy wraps another object and adds behaviour transparently before delegating.

```
1  const axiosInstance = axios.create({ baseURL, withCredentials:
    true });
2
3  axiosInstance.interceptors.request.use((config) => {
4    const token = localStorage.getItem("eats_token");
5    if (token) config.headers["Authorization"] = `Bearer
      ${token}`;
6    return config; // Callers never need to attach the token
      manually
7  });
```

Listing 9.6: Proxy Pattern in `axiosInstance.js`

9.7 7. Memento / Snapshot Pattern

Definition: The state of an object is captured at a moment in time and preserved independently of future changes to the original.

```
1  const enrichedItems = await Promise.all(
2    orderItems.map(async (item) => {
3      const doc = await MenuItem.findById(item.menuItemId);
4      return {
5        menuItemId:    item.menuItemId,
6        itemName:      doc?.menuItemName || item.itemName, //
          snapshot
7        itemPrice:     item.itemPrice,                      //
          snapshot
8        isVeg:         doc?.isMenuItemVeg ?? true,          //
          snapshot
9        itemQuantity: item.itemQuantity,
10     };
11   });
12 );
13 // Changes to the live menu do not alter historical order
    records.
```

Listing 9.7: Snapshot Pattern in `order.controller.ts`

9.8 8. Module Pattern

Definition: Related functions are grouped with a clean public interface; internal implementation details are hidden.

```
1  const TOKEN_KEY = "eats_token"; // private - not exported
2
3  export const authApi = {
4    login: async (credentials) => {
5      const res = await axiosInstance.post("/api/v1/auth/login",
6        credentials);
7      if (res.data.token) localStorage.setItem(TOKEN_KEY,
8        res.data.token);
9      return res.data;
10   },
11   logout: async () => {
12     localStorage.removeItem(TOKEN_KEY); // hidden from callers
13     await axiosInstance.post("/api/v1/auth/logout");
14   },
15   getProfile: async () => { /* ... */ },
16   updateProfile: async (formData) => { /* ... */ },
17 };
```

Listing 9.8: Module Pattern in `authApi.js`

9.9 9. Singleton Pattern

Definition: Only one instance of an object exists throughout the application; all parts share that instance.

Table 9.2: Singleton Instances in Eats

Singleton	Rationale for a Single Instance
axiosInstance	All API calls share one configured Axios instance with the base URL and interceptor
Redux Store	One central store shared by all components via <code><Provider store={appStore}></code>
MongoDB Connection	Mongoose maintains one internal connection pool; all queries reuse it
App class	One Express server created and started in <code>server.ts</code> for the process lifetime

10 SOLID Principles

SOLID is an acronym for five object-oriented design principles proposed by Robert C. Martin. Adhering to SOLID produces systems that are easier to maintain, extend, and test.

Table 10.1: SOLID Principles at a Glance

Letter	Principle	Core Rule
S	Single Responsibility	A class has exactly one reason to change
O	Open / Closed	Open for extension; closed for modification
L	Liskov Substitution	Subtypes are safely substitutable for their parent
I	Interface Segregation	No class is forced to implement unused methods
D	Dependency Inversion	High-level modules depend on abstractions, not concretions

10.1 S — Single Responsibility Principle

Every file in the backend has a single, well-defined purpose:

Table 10.2: SRP: One Responsibility Per File

File	Single Responsibility
<code>auth.controller.ts</code>	Handle HTTP requests and responses for authentication routes
<code>auth.service.ts</code>	Business logic: find user, compare password, generate token
<code>validation.ts</code>	Validate request data: email format, password strength
<code>userAuth.ts</code>	Verify JWT and attach the authenticated user to the request
<code>clouinary.ts</code>	Upload an in-memory buffer to Cloudinary; return the URL
<code>multer.ts</code>	Configure file upload middleware for multipart/form-data

10.2 O — Open/Closed Principle

The `App` class has never been modified to add a new feature domain. Adding support for notifications requires only one line in `server.ts`:

```
1 const app = new App([  
2   new AuthRoutes(),
```

```
3   new CartRoutes(),
4   new OrderRoutes(),
5   new PaymentRoutes(),
6   new RestaurantRoutes(),
7   // new NotificationRoutes(), <- add one line; App.ts
    unchanged
8 ];
```

Listing 10.1: OCP in `server.ts`

10.3 L — Liskov Substitution Principle

Any class satisfying the `Routes` interface can replace any other such class without breaking the `App` class:

```
1 private initializeRoutes(routes: Routes[]) {
2   routes.forEach((route) => {
3     this.app.use("/", route.router);
4     // AuthRoutes, CartRoutes, or any future Routes
        implementation
5     // all work identically here.
6   });
7 }
```

Listing 10.2: LSP in `app.ts`

10.4 I — Interface Segregation Principle

Interfaces are kept minimal. The `Routes` interface has only two fields (`path?` and `router`). Route classes are not obliged to implement database logic or email utilities. Each Mongoose model interface (`IUser`, `ICart`, `IOrder`) exposes only the methods relevant to that entity.

10.5 D — Dependency Inversion Principle

High-level modules depend on abstractions rather than on concrete implementations:

```
1 // App.ts imports the Routes interface, not AuthRoutes or
    CartRoutes
2 class App {
3   constructor(routes: Routes[]) { // abstraction, not a
        concrete class
4     this.initializeRoutes(routes);
5   }
}
```

```
6 }  
7 // server.ts is the composition root where concrete types are  
  supplied.
```

Listing 10.3: DIP in `app.ts`

11 Authentication Design

11.1 The Cross-Browser Problem

The original implementation stored the session token in an HTTP-only cookie. This failed silently in two environments:

- **Safari (Intelligent Tracking Prevention, ITP):** Apple blocks all cross-site cookies when the frontend (`vercel.app`) and backend (`onrender.com`) are on different top-level domains.
- **Chrome Incognito:** Third-party cookies are blocked by default since Chrome 120.

Because Vercel and Render operate on distinct domains, any cookie the backend sets is classified as a third-party cookie by these browsers, and is silently denied.

11.2 Solution: `localStorage` with the Authorization Header

Table 11.1: Authentication Solution Components

Component	Role
<code>authApi.js</code> (login)	Stores JWT in <code>localStorage</code> under key <code>"eats_token"</code> on success
<code>axiosInstance.js</code> (interceptor)	Reads <code>"eats_token"</code> and injects <code>Authorization: Bearer <token></code> before every request
<code>userAuth.ts</code> (middleware)	Reads token from the <code>Authorization</code> header first; falls back to cookie for compatible browsers
<code>userSlice.js</code> (logoutUser)	Calls <code>localStorage.removeItem("eats_token")</code> directly in the reducer

`localStorage` is governed by the same-origin policy, which is unaffected by third-party cookie restrictions. Authentication now works correctly across all browsers including Safari and Chrome Incognito.

11.3 Security Controls

Table 11.2: Security Controls

Control	Implementation
Password storage	Bcrypt with 10 salt rounds via a Mongoose pre-save hook
Credential enumeration	All authentication failures return the generic message “Invalid credentials”
Token expiry	<code>jwt.sign(..., { expiresIn: '7d' })</code>
Token removal on logout	<code>localStorage.removeItem()</code> called in the Redux reducer; guaranteed regardless of server state
Payment verification	Razorpay HMAC-SHA256 signature recomputed server-side and compared with the client-supplied value
CORS allowlist	Only the Vercel URL and localhost ports are whitelisted; all other origins are rejected
File upload validation	Multer restricts to <code>image/*</code> MIME types and a 5 MB maximum size
Password in responses	<code>delete userObject.password</code> called before every response containing user data
Environment secrets	All keys and URIs stored in <code>.env</code> files; never committed to the repository

12 Deployment

12.1 Deployment Topology

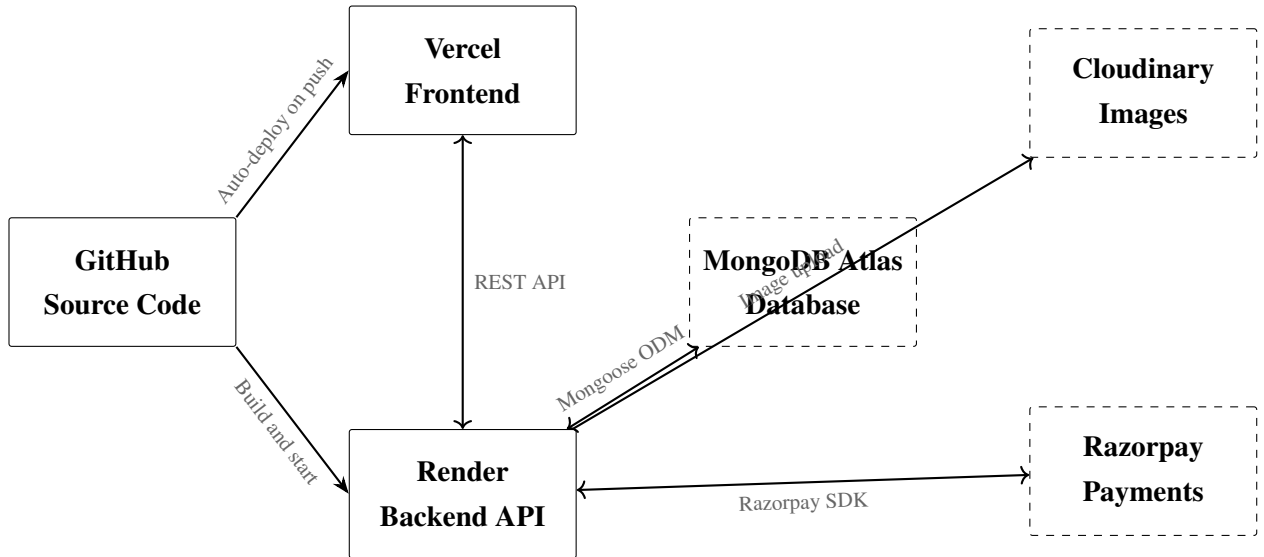


Figure 12.1: Production Deployment Architecture

12.2 Frontend — Vercel

Table 12.1: Vercel Configuration

Setting	Value
Platform	Vercel
Root directory	client/
Build command	npm run build (Vite)
Output directory	dist/ (auto-detected)
Environment	VITE_BACKEND_URL = Render backend URL
HTTPS	Automatic; Vercel manages TLS certificates

12.3 Backend — Render

Table 12.2: Render Configuration

Setting	Value
Platform	Render (Web Service)
Root directory	server/
Build command	npm install --include=dev && npm run build
Start command	npm start (runs node dist/server.js)
Runtime	Node.js

Why compile TypeScript at build time? Running `ts-node` at runtime on Render’s 512 MB free tier causes out-of-memory failures. The TypeScript compiler (`tsc`) transforms the entire codebase to plain JavaScript during the build step. The resulting `dist/server.js` runs efficiently without any runtime compilation overhead.

12.4 Environment Variables

Table 12.3: Server Environment Variables (`server/.env`)

Variable	Required	Purpose
PORT	No	HTTP port; defaults to 8080
MONGODB_URI	Yes	Full MongoDB Atlas connection string
JWT_SECRET	Yes	Secret key for signing and verifying JWTs
CLOUDINARY_CLOUD_NAME	Yes	Cloudinary account identifier
CLOUDINARY_API_KEY	Yes	Cloudinary API key
CLOUDINARY_API_SECRET	Yes	Cloudinary API secret
RAZORPAY_KEY_ID	Yes	Razorpay public key ID
RAZORPAY_KEY_SECRET	Yes	Razorpay secret for HMAC-SHA256 verification
NODE_ENV	Yes	Set to <code>production</code> on Render; controls cookie security flags

13 Limitations and Future Scope

13.1 Current Limitations

Table 13.1: Known Limitations

Feature	Current State
Payment	Simulated; the Razorpay UI is shown but no real transaction is processed
Reorder button	Visible in the order history UI but not connected to any backend action
Navbar search	Accepts text input but does not filter restaurants or navigate to results
Sort by cost	Sorts on the rating field; no price-range field exists in the Restaurant schema
Admin panel	No interface for adding or managing restaurant and menu data
Order tracking	Client-side countdown only; the server does not push live status updates

13.2 Future Enhancements

Table 13.2: Planned Enhancements

Enhancement	Description
Live Razorpay integration	Full payment flow with real Razorpay order, SDK modal, and webhook verification
WebSocket order tracking	Socket.io server-pushed order status updates in real time
Admin dashboard	CRUD interface for restaurants and menu items with image upload
Full-text search	MongoDB Atlas Search (Lucene-backed) for restaurant and menu discovery
Email notifications	Order confirmation and status emails via Nodemailer or Send-Grid
Ratings and reviews	User ratings on orders; aggregated scores updated in the Restaurant document
Progressive Web App	Service worker, offline caching, and a home-screen install prompt
Geo-based discovery	MongoDB <code>\$near</code> geospatial queries to surface nearby restaurants

14 Conclusion

14.1 Summary

Eats is a production-deployed, full-stack food delivery application that moves well beyond a tutorial project. It engages real engineering problems found in production systems:

- The backend enforces a **five-layer class-based TypeScript architecture** with clear separation of concerns at every layer
- All five **SOLID principles** are applied throughout the codebase with verifiable code evidence
- **Nine design patterns** are used, each arising naturally from a specific engineering problem
- The **cross-browser authentication failure** caused by Safari ITP was diagnosed and resolved using a `localStorage` plus `Authorization` header approach
- The **debounced cart sync** with a logout-transition guard prevents both database spam and accidental data loss on logout
- An **order snapshot mechanism** ensures historical order records remain accurate regardless of future menu changes

14.2 Key Engineering Decisions

Table 14.1: Engineering Decisions and Their Rationale

Decision	Rationale
TypeScript over plain JavaScript	Static typing catches errors at compile time; enables OOP with access modifiers and interfaces
Five-layer class-based backend	Clear responsibility boundaries; mirrors patterns in NestJS and Spring Boot
JWT in localStorage	Avoids third-party cookie blocking by Safari ITP and Chrome Incognito in cross-domain deployments
Debounced cart sync	Prevents flooding MongoDB with writes on every user interaction
Order item snapshots	Decouples historical records from live menu data; preserves prices and names at purchase time
Buffer-to-stream image upload	Eliminates temporary disk writes; simplifies cleanup and reduces I/O overhead
Compile TypeScript at build time	Avoids <code>ts-node</code> runtime overhead and out-of-memory failures on Render's free tier

14.3 Learning Outcomes

This project provided practical experience with:

1. Full-stack MERN development with TypeScript, from design through production deployment
2. Object-oriented programming and class-based server architecture
3. SOLID principles applied to real production code
4. Nine design patterns identified, justified, and implemented
5. Cross-browser authentication diagnosis and remediation
6. Integration of cloud services: MongoDB Atlas, Cloudinary, Razorpay
7. Redux state management including slices, selectors, and store subscribers
8. Performance engineering: lazy loading, debouncing, race-condition prevention, connection pooling

References

- [1] Martin, R. C. (2003). *Agile Software Development, Principles, Patterns, and Practices*. Pearson Education.
- [2] Gamma, E., Helm, R., Johnson, R., & Vlissides, J. (1994). *Design Patterns: Elements of Reusable Object-Oriented Software*. Addison-Wesley.
- [3] MongoDB Inc. (2026). *Data Modeling Introduction*. Retrieved from <https://www.mongodb.com/docs/>
- [4] Express.js Contributors. (2026). *Express 5.x API Reference*. Retrieved from <https://expressjs.com>
- [5] Meta Platforms. (2026). *React Documentation*. Retrieved from <https://react.dev>
- [6] Redux Contributors. (2026). *Redux Toolkit Documentation*. Retrieved from <https://redux-toolkit.js.org>
- [7] Auth0 by Okta. (2026). *Introduction to JSON Web Tokens*. Retrieved from <https://jwt.io/introduction>
- [8] Cloudinary Ltd. (2026). *Node.js SDK — Upload API*. Retrieved from <https://cloudinary.com/documentation>
- [9] Razorpay Software Pvt. Ltd. (2026). *Payment Gateway Integration*. Retrieved from <https://razorpay.com/docs>
- [10] Vercel Inc. (2026). *Deploying Vite Projects*. Retrieved from <https://vercel.com/docs>
- [11] Microsoft. (2026). *TypeScript Handbook*. Retrieved from <https://www.typescriptlang.org/docs>
- [12] MDN Web Docs. (2026). *Intersection Observer API*. Retrieved from <https://developer.mozilla.org>
- [13] Apple WebKit Team. (2023). *Intelligent Tracking Prevention*. Retrieved from <https://webkit.org/tracking-prevention>